INTRODUCTION TO DEEP LEARNING

Course number: 00240332

Lecture 3: Regression and Classification

Xiaolin Hu (胡晓林)

Dept. of Computer Science and Technology

Tsinghua University

Last lecture review

I. Math basics

PMF

PDF

Joint Prob

Marginal prob

Conditional prob

Gradient decent



II. Machine learning basics

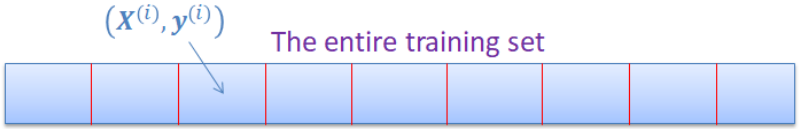
- Task T
- Performance P
- Experience E

Model capacity

MLE

$$\boldsymbol{\theta}_{\mathrm{ML}} = \arg\max_{\boldsymbol{\theta}} p_{\mathrm{model}}(\mathbb{X}; \boldsymbol{\theta})$$

SGD



Outline

I. Logistic regression

II. Softmax regression

III. Summary

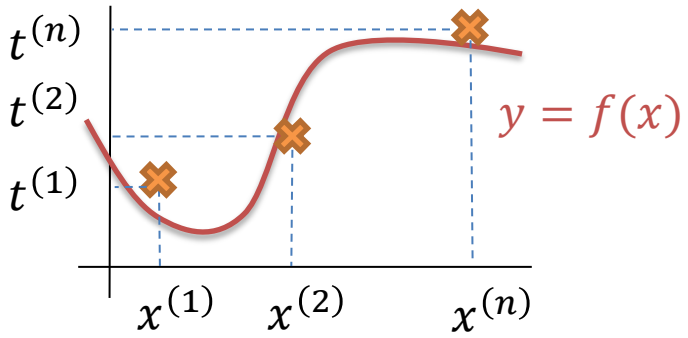
Regression and classification

Given a set of data points $\boldsymbol{x}^{(n)} \in R^m$ and the corresponding labels $t^{(n)} \in \Omega$: $\{(\boldsymbol{x}^{(1)}, t^{(1)}), (\boldsymbol{x}^{(2)}, t^{(2)}), \ldots, (\boldsymbol{x}^{(N)}, t^{(N)})\}$, for a new data point $\boldsymbol{x}$, predict its label

The goal is to find a mapping $f: R^m \rightarrow \Omega$

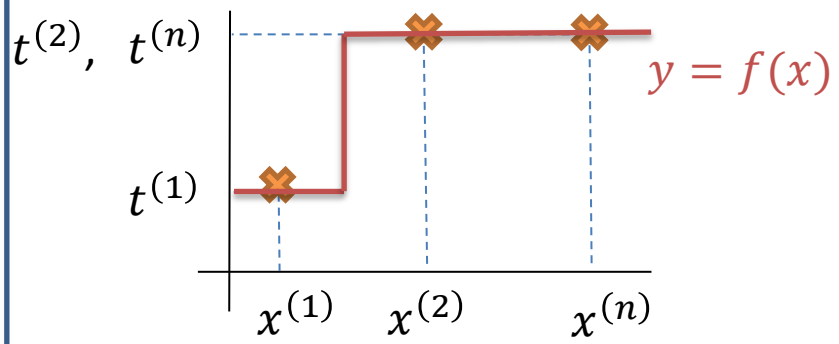
Regression

Ω is continuous



Classification

Ω is discrete

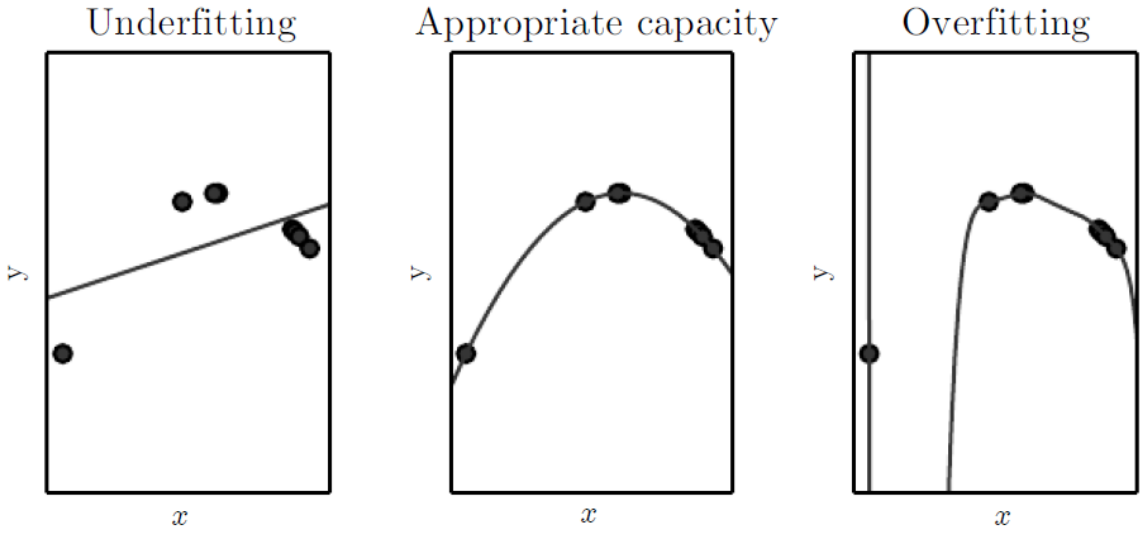


Recall: polynomial regression

- Consider a regression problem in which the input x and output y are both scalars. Find a function $f: \mathbb{R} \to \mathbb{R}$ to fit the data
  - $f(x) = b + wx$
  - $f(x) = b + w_1 x + w_2 x^2$
  - $f(x) = b + \sum_{i=1}^{9} w_i x^i$

MSE training:

$$\min_w \frac{1}{N} \sum_{n=1}^{N} \left|\left| f\left(x^{(n)}\right) - y^{(n)} \right|\right|_2^2$$



Linear regression

- $f(\boldsymbol{x})$ is linear

$$f(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x} + b$$

bias

b can absorbed into a new vector θ and $f(\boldsymbol{x}) = \boldsymbol{\theta}^\top \boldsymbol{x}$

where $\boldsymbol{w} \in R^m, b \in R$.

- Choose the cost function as the mean squared error (MSE)

$$E = \frac{1}{2N}\sum_{n=1}^{N}\left(f\left(\boldsymbol{x}^{(n)}\right) - t^{(n)}\right)^2 = \frac{1}{2N}\sum_{n=1}^{N}\left(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b - t^{(n)}\right)^2$$

- Find optimal $\boldsymbol{w}^*$ and $\boldsymbol{b}^*$ by minimizing the cost function

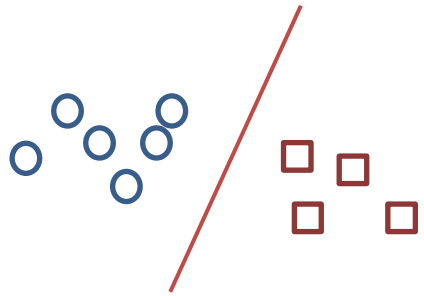
$$\nabla_{\boldsymbol{w}} E = \frac{1}{N}\sum_{n=1}^{N}\left(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b - t^{(n)}\right)\boldsymbol{x}^{(n)} = 0$$

$$\nabla_b E = \frac{1}{N}\sum_{n=1}^{N}\left(\boldsymbol{w}^\top \boldsymbol{x}^{(n)} + b - t^{(n)}\right) = 0$$

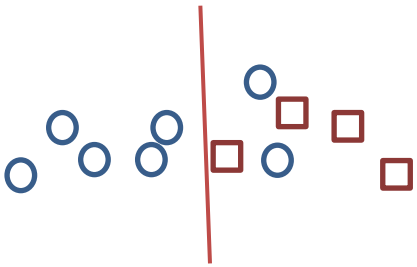
$\boldsymbol{w}^*, b^*$

Linear classification

- In the feature space, a linear classifier corresponds to a hyperplane



Linearly separable

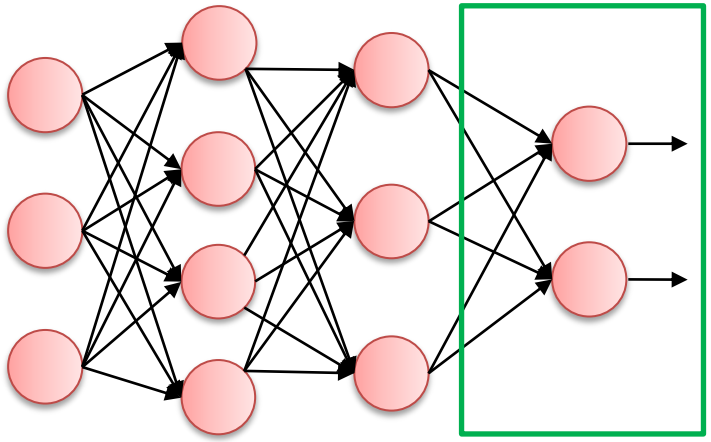


Linearly nonseparable

- Two typical linear classifiers
  - Perceptron
  - Support vector machine (SVM)

Discussion on classifiers

- What else classifiers do you know?
- If a neural network is used for classification, which classifier do you want to use in the output layer?

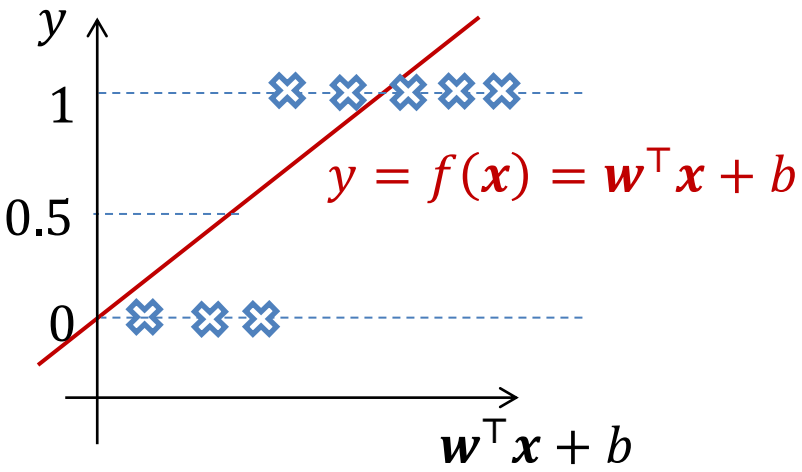


Any challenge here?

What shall we do?

Do binary classification using linear regression

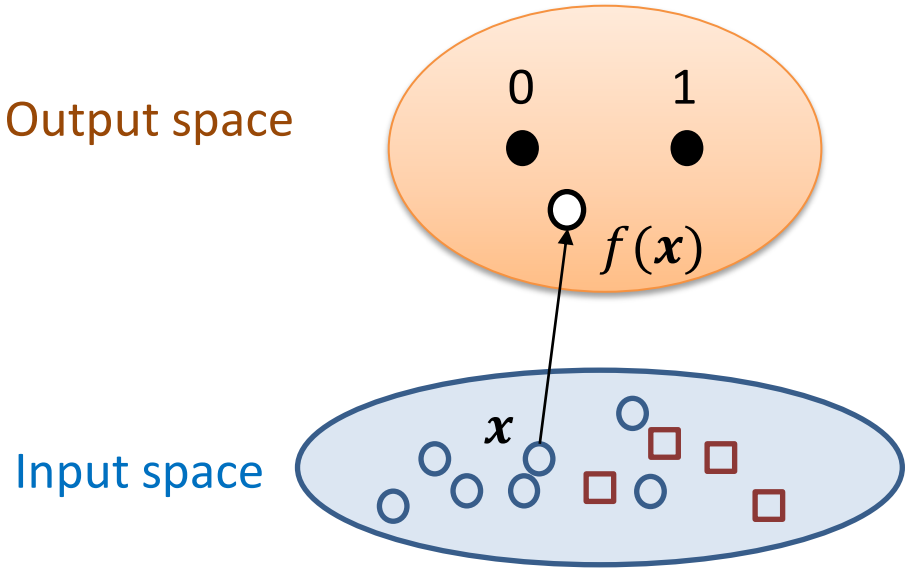
- Suppose $t \in \{0,1\}$. Consider $\boldsymbol{x} \in R^m$, can we use linear regression to do binary classification?



- Regression
  - Prediction $y = f(\boldsymbol{x})$ which is continuous
- Classification
  - Prediction $y = \begin{cases} 1, \text{if } f(\boldsymbol{x}) \geq 0.5 \\ 0, \text{if } f(\boldsymbol{x}) < 0.5 \end{cases}$

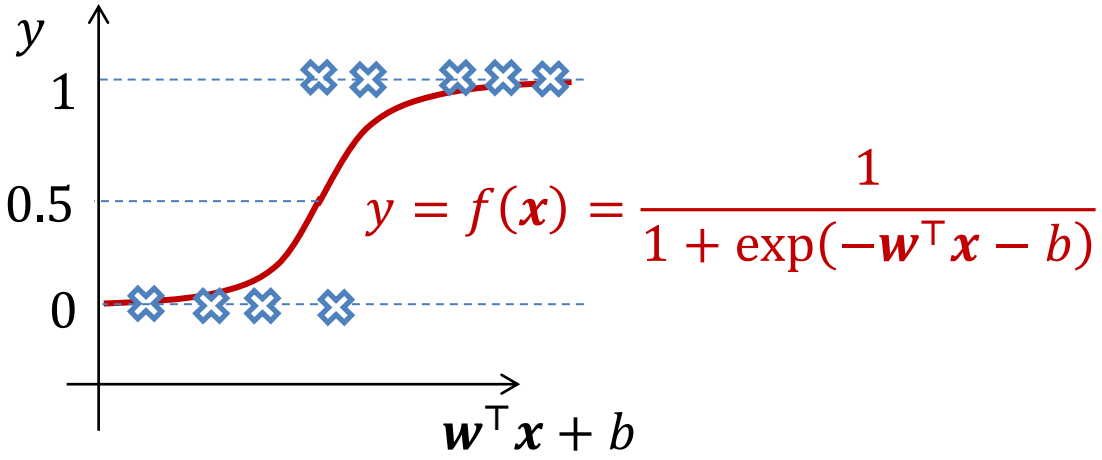
Input-output mapping

Use a linear function $f(\boldsymbol{x}) = \boldsymbol{w}^{\top}\boldsymbol{x} + b$ to map the input $\boldsymbol{x} \in R^m$ to the 1D output space $\{0,1\}$



Do binary classification using nonlinear regression

- $f(x)$ can be a nonlinear functions, e.g., the logistic sigmoid function



- Regression
  - Prediction $y = f(\boldsymbol{x})$ which is continuous
- Classification
  - Prediction $y = \begin{cases} 1, \text{if } f(\boldsymbol{x}) \geq 0.5 \\ 0, \text{if } f(\boldsymbol{x}) < 0.5 \end{cases}$

Train the nonlinear regression model

- $f(\boldsymbol{x})$ is nonlinear

$$f(\boldsymbol{x}) = h(\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + b)$$

  where $\boldsymbol{w} \in R^m, b \in R$.

- Choose the cost function as the mean squared error (MSE)

$$E = \frac{1}{2N}\sum_{n=1}^{N}\left(f\left(\boldsymbol{x}^{(n)}\right) - t^{(n)}\right)^2$$

- Find optimal $\boldsymbol{w}^*$ and $\boldsymbol{b}^*$ by minimizing the cost function

$$\nabla_{\boldsymbol{w}} E = \frac{1}{N}\sum_{n=1}^{N}\left(f\left(\boldsymbol{x}^{(n)}\right) - t^{(n)}\right) f'(\cdot)\boldsymbol{x}^{(n)} = 0$$

$$\nabla_b E = \frac{1}{N}\sum_{n=1}^{N}\left(f\left(\boldsymbol{x}^{(n)}\right) - t^{(n)}\right) f'(\cdot) = 0$$

$\boldsymbol{w}^*, b^*$

Normal distribution assumption

- Assume the label follows a normal distribution with mean $f(\mathbf{x}) = h(\boldsymbol{w}^{\mathsf{T}}\mathbf{x} + b)$:

$$p(\mathrm{t}|\mathbf{x}) = \mathcal{N}(\mathrm{t}; f(\mathbf{x})) \propto \exp\left(-\left(\mathrm{t} - f(\mathbf{x})\right)^2\right)$$

(sometimes we may not distinguish between plain and italic type faces)

- Given a dataset $\left\{\left(\boldsymbol{x}^{(1)}, t^{(1)}\right), \ldots, \left(\boldsymbol{x}^{(N)}, t^{(N)}\right)\right\}$. View t and $\mathbf{x}$ as random variables. The conditional data likelihood function (independence assumption)

$$P\left(t^{(1)}, \ldots, t^{(N)} \middle| \boldsymbol{X}\right) = \Pi_{n=1}^{N} P\left(t^{(n)} | \boldsymbol{x}^{(n)}\right)$$

- $\max \log P\left(t^{(1)}, \ldots, t^{(N)} \middle| \boldsymbol{X}\right)$ is equivalent to

$$\min E = \frac{1}{2N} \sum_{n=1}^{N} \left(f\left(\boldsymbol{x}^{(n)}\right) - t^{(n)}\right)^2$$

Bernoulli distribution assumption for 2-class classification

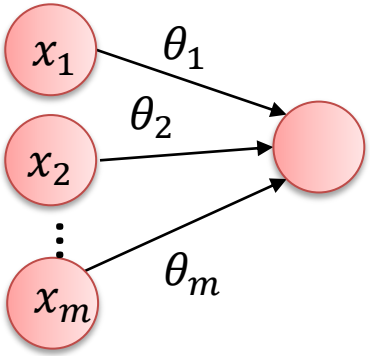
$$p(\mathrm{t}|\mathbf{x}) = \mathcal{N}(\mathrm{t}; f(\mathbf{x})) \propto \exp\left(-\left(\mathrm{t} - f(\mathbf{x})\right)^2\right)$$

- For regression (t is continuous), the normal distribution assumption is natural
- For classification (t is discrete), it is strange
- We have more suitable assumptions on the data distribution for classification
  - Bernoulli distribution

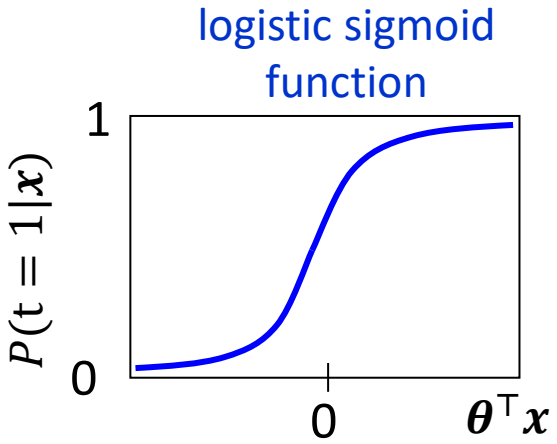
To do so, we need to first write down the Bernoulli distribution with (parameterized) probability $P(t = 1|\boldsymbol{x})$

Logistic regression

- For 2-class problems, one 0-1 unit is enough for representing a label



$t^{(1)}$	$t^{(2)}$	$t^{(3)}$	…
1	0	1	…



- We try to learn a conditional probability (we've absorbed b in θ)

$$P(\mathrm{t}=1|\boldsymbol{x}) = \frac{1}{1+\exp(-\theta^\top \boldsymbol{x})} \triangleq h(\boldsymbol{x})$$

$$P(\mathrm{t}=0|\boldsymbol{x}) = 1 - P(t=1|\boldsymbol{x}) = 1 - h(\boldsymbol{x})$$

where $\boldsymbol{x}$ is input and t is label

$P(\mathrm{t}=1|\boldsymbol{x})$ is a Bernoulli distribution

Maximum conditional data likelihood

- *Recall* the maximum conditional likelihood estimation:
  - 1. write down the conditional likelihood function
  - 2. take log and maximize
- Given a dataset $\{(\boldsymbol{x}^{(1)}, t^{(1)}), \ldots, (\boldsymbol{x}^{(N)}, t^{(N)})\}$ where $t^{(n)} \in \{0,1\}$
- View t as a Bernoulli variable and $P(\mathrm{t} = 1|\boldsymbol{x}) = h(\boldsymbol{x}; \boldsymbol{\theta})$. The conditional likelihood function

$$P(t^{(1)}, \ldots, t^{(N)}|X; \boldsymbol{\theta}) = \Pi_{n=1}^{N} h(\boldsymbol{x}^{(n)})^{t^{(n)}} (1 - h(\boldsymbol{x}^{(n)}))^{1-t^{(n)}}$$

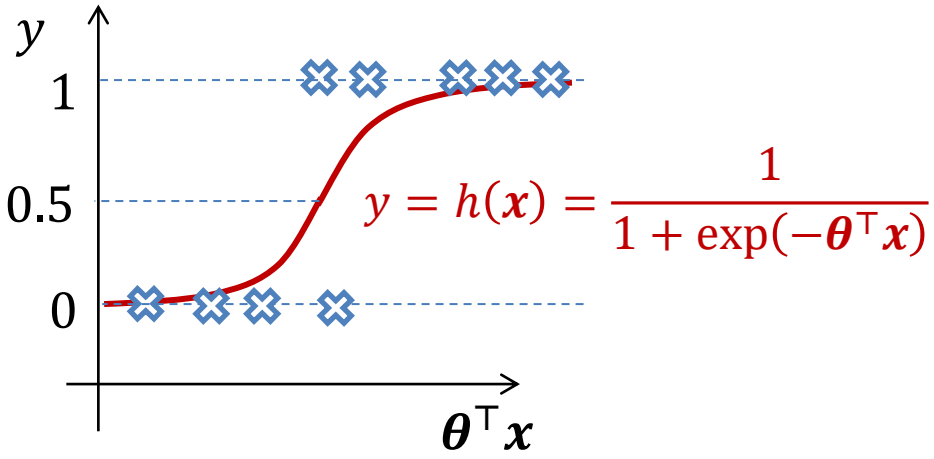
- Maximizing the likelihood is equivalent to minimizing

$$E(\boldsymbol{\theta}) = -\frac{1}{N} \ln P(t^{(1)}, \ldots, t^{(N)})$$

Cross-entropy (CE) function

$$= -\frac{1}{N} \sum_{n=1}^{N} \left(t^{(n)} \ln h(\boldsymbol{x}^{(n)}) + (1 - t^{(n)}) \ln(1 - h(\boldsymbol{x}^{(n)})) \right)$$

Sigmoid classifier with different loss functions



- Mean squared error (MSE) function

$$E(\boldsymbol{\theta}) = \frac{1}{2N}\sum_{n=1}^{N}\left(h\left(\boldsymbol{x}^{(n)}\right) - t^{(n)}\right)^2$$

- Cross-entropy function

$$E(\boldsymbol{\theta}) = -\frac{1}{N}\sum_{n=1}^{N}\left(t^{(n)}\ln h(\boldsymbol{x}^{(n)}) + (1-t^{(n)})\ln(1-h(\boldsymbol{x}^{(n)}))\right)$$

$h(\boldsymbol{x})$ is interpreted as a probability

Exercise: Calculate the gradient

Cross-entropy (CE) function

$$E(\boldsymbol{\theta}) = -\frac{1}{N}\sum_{n=1}^{N}\left(t^{(n)}\ln h(\boldsymbol{x}^{(n)}) + (1-t^{(n)})\ln(1-h(\boldsymbol{x}^{(n)}))\right)$$

where $h(\boldsymbol{x}) = \dfrac{1}{1+\exp(-\boldsymbol{\theta}^{\top}\boldsymbol{x})}$

$\nabla E(\boldsymbol{\theta}) = ?$

$$h(z) = \frac{1}{1+\exp(-z)}$$

$$h' = h(1-h)$$

Exercise: Calculate the gradient

Cross-entropy (CE) function

$$E(\boldsymbol{\theta}) = -\frac{1}{N}\sum_{n=1}^{N}\left(t^{(n)}\ln h(\boldsymbol{x}^{(n)}) + (1-t^{(n)})\ln(1-h(\boldsymbol{x}^{(n)}))\right)$$

where $h(\boldsymbol{x}) = \dfrac{1}{1+\exp(-\boldsymbol{\theta}^\top \boldsymbol{x})}$

$\nabla E(\boldsymbol{\theta}) = ?$

$$h(z) = \frac{1}{1+\exp(-z)}$$

$$h' = h(1-h)$$

Answer: $\nabla E(\boldsymbol{\theta}) = \dfrac{1}{N}\sum_{n}\boldsymbol{x}^{(n)}\left(h(\boldsymbol{x}^{(n)}) - t^{(n)}\right)$

投票 最多可选1项

Setting

Is your result correct?

A Yes

B No

Submit

单选题 1分

设置

What’s the minimum of the CE function

$$E(\boldsymbol{\theta}) = -\frac{1}{N}\ln P(t^{(1)},\ldots,t^{(N)})$$

$$= -\frac{1}{N}\sum_{n=1}^{N}\left(t^{(n)}\ln h(\boldsymbol{x}^{(n)}) + (1-t^{(n)})\ln(1-h(\boldsymbol{x}^{(n)}))\right)$$

A 0

B 1

C $-\infty$

提交

Training and testing

$$E(\boldsymbol{\theta}) = -\frac{1}{N}\sum_{n=1}^{N}\left(t^{(n)}\ln h(\boldsymbol{x}^{(n)}) + (1-t^{(n)})\ln(1-h(\boldsymbol{x}^{(n)}))\right)$$

- Calculate the gradient

$$\nabla E(\boldsymbol{\theta}) = \frac{1}{N}\sum_{n}\boldsymbol{x}^{(n)}\left(h(\boldsymbol{x}^{(n)}) - t^{(n)}\right)$$

- Some regularization term can be incorporated into the cost function

$$J(\boldsymbol{\theta}) = E(\boldsymbol{\theta}) + \lambda\left|\left|\boldsymbol{\theta}\right|\right|^2/2$$

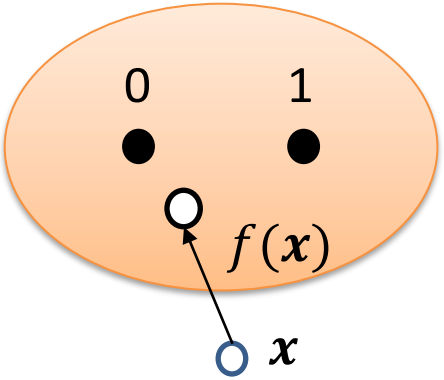
- Training: learn $\boldsymbol{\theta}$ to minimize the cost function

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha\nabla J(\boldsymbol{\theta})$$

  where α is the learning rate
- Testing: for a new input $\boldsymbol{x}$, if $P(t=1|\boldsymbol{x}) > P(t=0|\boldsymbol{x})$ then we predict the input as class 1, and 0 otherwise

Summary of part 1

Function mapping



Linear regression → nonlinear regression

$$\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{W}^{\top}\boldsymbol{x} + \boldsymbol{b} \qquad \boldsymbol{f}(x) = \boldsymbol{g}(\boldsymbol{W}^{\top}\boldsymbol{x} + \boldsymbol{b})$$

MLE

1. normal distribution:
   - $\boldsymbol{g}(\cdot)$ is any nonlinear function
   - Loss function: MSE
2. Bernoulli distribution:
   - $\boldsymbol{g}(\cdot)$ is sigmoid function
   - Loss function: Cross-entropy

Logistic regression!

Outline

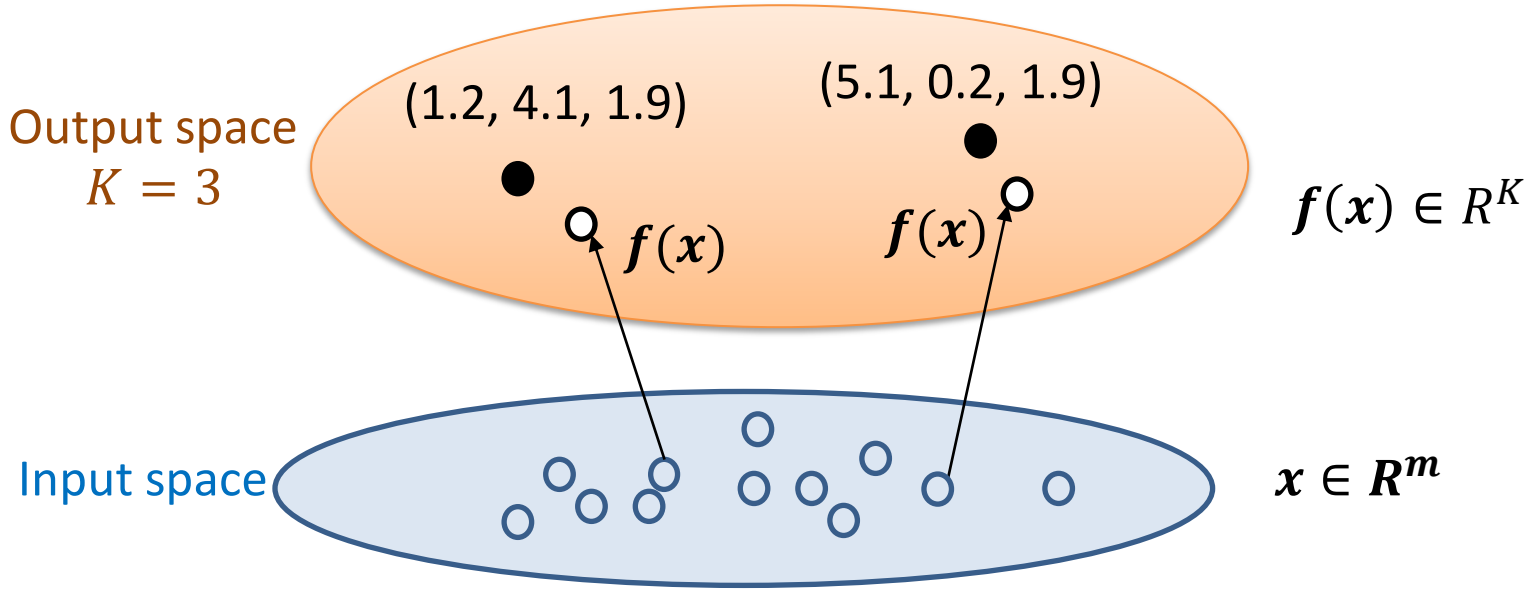
I. Logistic regression

II. Softmax regression

III. Summary

Linear regression for vectors

- If $t \in R^K$ is a continuous vector, then use a linear function $f_k(\boldsymbol{x})$ to regress t_k for $k = 1, \ldots, K$: $f_k(\boldsymbol{x}) = \boldsymbol{w}_k^\top \boldsymbol{x} + b_k$, where $\boldsymbol{w_k} \in R^m, b_k \in R$



Linear regression for vectors

- $f_k(\boldsymbol{x})$ is linear for $k = 1, \ldots, K$: $f_k(\boldsymbol{x}) = \boldsymbol{w}_k^\top \boldsymbol{x} + b_k$, where $\boldsymbol{w_k} \in R^m, b_k \in R$.
- Choose the cost function as the mean squared error (MSE)

$$E = \frac{1}{2N} \sum_{n=1}^{N} \sum_{k=1}^{K} \left(f_k\left(\boldsymbol{x}^{(n)}\right) - t_k^{(n)} \right)^2$$

- Find optimal $\boldsymbol{w}_k^*$ and $\boldsymbol{b}_k^*$ by solving the linear system

$$\nabla_{\boldsymbol{w_k}} E = \frac{1}{N} \sum_{n=1}^{N} \left(\boldsymbol{w}_k^\top \boldsymbol{x}^{(n)} + b_k - t_k^{(n)} \right) \boldsymbol{x}^{(n)} = 0$$

$$\nabla_{b_k} E = \frac{1}{N} \sum_{n=1}^{N} \left(\boldsymbol{w}_k^\top \boldsymbol{x}_k^{(n)} + b_k - t_k^{(n)} \right) = 0$$

$\boldsymbol{w}_k^*, b_k^*$

Nonlinear regression for vectors

- $f_k(\boldsymbol{x})$ is nonlinear for $k = 1, \ldots, K$

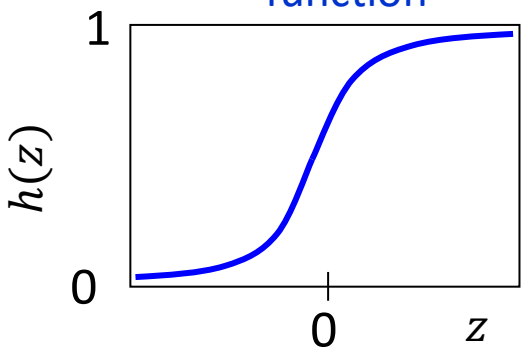
$$f_k(\boldsymbol{x}) = h\left(\boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{x} + b_k\right)$$

where $\boldsymbol{w_k} \in R^m, b_k \in R$, and

$$h(z) = \frac{1}{1 + \exp(-z)}$$

Or other functions

logistic sigmoid function



- Choose the cost function as the MSE

$$E = \frac{1}{N}\sum_{n=1}^{N} E^{(n)}$$

where $E^{(n)} = \frac{1}{2}\sum_{k=1}^{K}\left(f_k\left(\boldsymbol{x}^{(n)}\right) - t_k^{(n)}\right)^2$

Gradient of the cost function

$$E = \frac{1}{N}\sum_{n=1}^{N} E^{(n)} \text{ where } E^{(n)} = \frac{1}{2}\sum_{k=1}^{K}\left(f_k\left(\boldsymbol{x}^{(n)}\right) - t_k^{(n)}\right)^2$$

- Denote $u_k^{(n)} = \boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{x}^{(n)} + b_k$ and $\delta_k^{(n)} = \frac{\partial E^{(n)}}{\partial u_k^{(n)}}$, then

$$\frac{\partial E^{(n)}}{\partial \boldsymbol{w}_k} = \delta_k^{(n)}\boldsymbol{x}^{(n)}$$

$$\frac{\partial E^{(n)}}{\partial b_k} = \delta_k^{(n)}\frac{\partial u_k^{(n)}}{\partial b_k} = \delta_k^{(n)}$$

$\delta_k^{(n)}$: Local sensitivity or local gradient

$$\delta_k^{(n)} = \left(f(u_k^{(n)}) - t_k\right) f'(u_k^{(n)})$$

Vector-matrix form

- Define

$$\boldsymbol{W} = \begin{pmatrix} w_{11} & \cdots & w_{1m} \\ \vdots & \vdots & \vdots \\ w_{K1} & \cdots & w_{Km} \end{pmatrix} \quad \frac{\partial E}{\partial \boldsymbol{W}} = \begin{pmatrix} \partial E/\partial w_{11} & \cdots & \partial E/\partial w_{1m} \\ \vdots & \vdots & \vdots \\ \partial E/\partial w_{K1} & \cdots & \partial E/\partial w_{Km} \end{pmatrix}$$

m: the number of inputs; K: the number of outputs

- Output: $\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{h}(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b})$, where $\boldsymbol{f}, \boldsymbol{h}, \boldsymbol{b} \in R^K$, $\boldsymbol{x} \in R^m$
- Error function: $E = \frac{1}{2N}\sum_{n=1}^{N} \left|\left| \boldsymbol{f}\left(\boldsymbol{x}^{(n)}\right) - \boldsymbol{t}^{(n)} \right|\right|_2^2$
- Gradient

Elementwise product

$$\nabla_{\boldsymbol{W}} E = \frac{1}{N}\sum_{n=1}^{N} \left(\boldsymbol{f}\left(\boldsymbol{x}^{(n)}\right) - \boldsymbol{t}^{(n)}\right) \odot \boldsymbol{f}'\left(\boldsymbol{x}^{(n)}\right) \left(\boldsymbol{x}^{(n)}\right)^{\mathsf{T}} \in R^{K \times m}$$

$$\nabla_{\boldsymbol{b}} E = \frac{1}{N}\sum_{n=1}^{N} \left(\boldsymbol{f}\left(\boldsymbol{x}^{(n)}\right) - \boldsymbol{t}^{(n)}\right) \odot \boldsymbol{f}'\left(\boldsymbol{x}^{(n)}\right) \in R^{K}$$

Representation of class labels

- For classification, given $\{(\boldsymbol{x}^{(1)}, t^{(1)}), \ldots, (\boldsymbol{x}^{(N)}, t^{(N)})\}$, the goal is to find a mapping from $\boldsymbol{x}^{(n)}$ to $t^{(n)}$

$$f: R^m \rightarrow \Omega$$

  where Ω is a discrete set

- $t^{(n)}$ can be a (discrete) scalar or vector

Suppose there are 5 classes in total

Rarely used

Scalar representation

$t^{(1)} = 1$

$t^{(3)} = 3$

Vector representation

$\boldsymbol{t}^{(1)} = (1, 0, 0, 0, 0)^{\mathsf{T}}$

$\boldsymbol{t}^{(3)} = (0, 0, 1\ 0, 0)^{\mathsf{T}}$

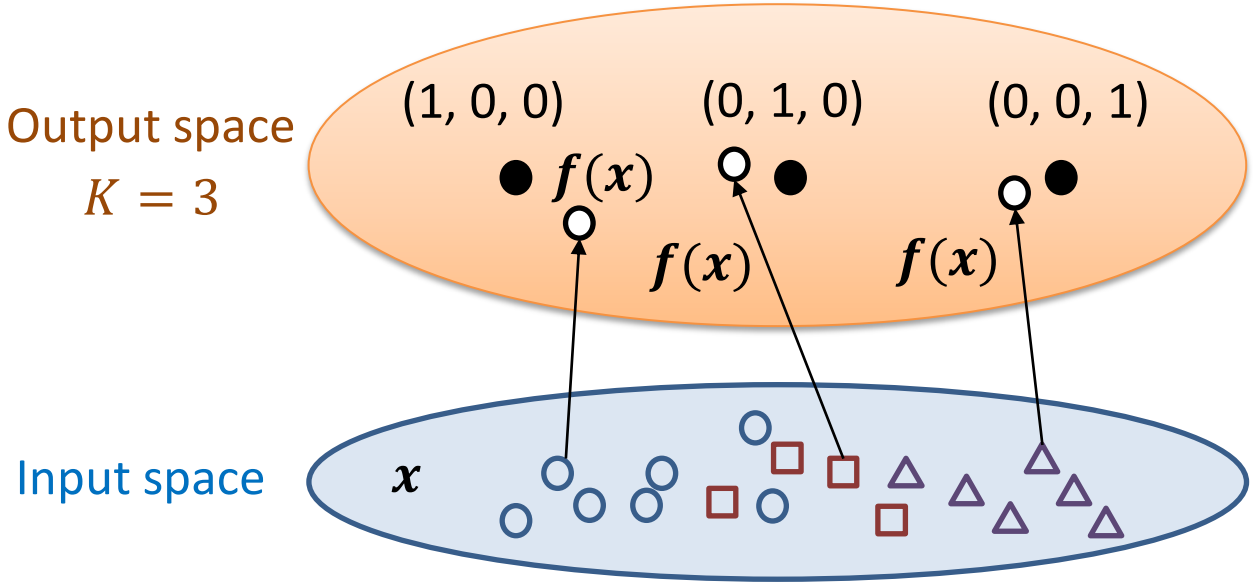
Usually used

Any problem with scalar representation?

- 1-of-K representation
- Property: $t_k^{(n)} \in \{0,1\}$; $\sum_k t_k^{(n)} = 1$

Do multilabel classification using regression

- Using the 1-of-K representation for class labels, one can also do classification using linear regression



How to obtain the class label considering that $f(x)$ is real?

- Both linear and nonlinear regression discussed before can be applied

Normal distribution assumption

- Assume the label follows a normal distribution with mean $\boldsymbol{f}(\mathbf{x}) = \boldsymbol{h}(\boldsymbol{W}\mathbf{x} + \boldsymbol{b})$:

$$p(\mathbf{t}|\mathbf{x}) = \mathcal{N}(\mathbf{t}; \boldsymbol{f}(\mathbf{x}), \boldsymbol{I}) \propto \exp\left(-\left|\left|\mathbf{t} - \boldsymbol{f}(\mathbf{x})\right|\right|_2^2\right)$$

(sometimes we may not distinguish between plain and italic type faces)

- Given a dataset $\left\{\left(\boldsymbol{x}^{(1)}, \boldsymbol{t}^{(1)}\right), \ldots, \left(\boldsymbol{x}^{(N)}, \boldsymbol{t}^{(N)}\right)\right\}$. View $\mathbf{t}$ and $\mathbf{x}$ as random variables. The conditional data likelihood function (independence assumption)

$$P\left(\boldsymbol{t}^{(1)}, \ldots, \boldsymbol{t}^{(N)} \middle| \boldsymbol{X}\right) = \Pi_{n=1}^{N} P(\boldsymbol{t}^{(n)} | \boldsymbol{x}^{(n)})$$

- $\max \log P\left(\boldsymbol{t}^{(1)}, \ldots, \boldsymbol{t}^{(N)} \middle| \boldsymbol{X}\right)$ is equivalent to

$$\min E = \frac{1}{2N} \sum_{n=1}^{N} \left|\left|\boldsymbol{f}\left(\boldsymbol{x}^{(n)}\right) - \boldsymbol{t}^{(n)}\right|\right|_2^2$$

Multinoulli distribution assumption for classification

$$p(\mathbf{t}|\mathbf{x}) = \mathcal{N}(\mathbf{t}; \boldsymbol{f}(\mathbf{x}), \boldsymbol{I}) \propto \exp\left(-\left|\left|\mathbf{t} - \boldsymbol{f}(\mathbf{x})\right|\right|_2^2\right)$$

- For regression (t is continuous), the normal distribution assumption is natural
- For classification (t is discrete), it is strange
- We have more reasonable assumptions on the data distribution for classification
  - Bernoulli distribution for $K = 2$
  - Multinoulli or categorical distribution for $K > 2$

To do so, we need to first write down the multinoulli distribution with (parameterized) probability $P(t_k = 1|\boldsymbol{x})$

Recap: multinoulli or categorical prob distributions

- The prob distribution over a single discrete variable t with K different states where K is finite

$$P(\mathrm{t} = k|\boldsymbol{p}) = p_k$$

where $\boldsymbol{p} \in [0,1]^K$ and $\sum_{k=1}^{K} p_k = 1$

- With one-hot representation $\mathbf{t} = (0, \dots, 1, \dots, 0)^\top$, then $P(\mathbf{t}) = \Pi_{k=1}^{K} P(\mathrm{t}_k = 1)^{t_k}$

A random variable

A value 0 or 1

Scalar: 1,2,3

$P(\mathrm{t} = 1) = 0.2;$
$P(\mathrm{t} = 2) = 0.5;$
$P(\mathrm{t} = 3) = 0.3$

One-hot: $(1,0,0)^\top, (0,1,0)^\top, (0,0,1)^\top$

$P(\mathbf{t} = (1,0,0)^\top)$
$= P(\mathrm{t}_1 = 1)^1 P(\mathrm{t}_2 = 1)^0 P(\mathrm{t}_3 = 1)^0 = 0.2$

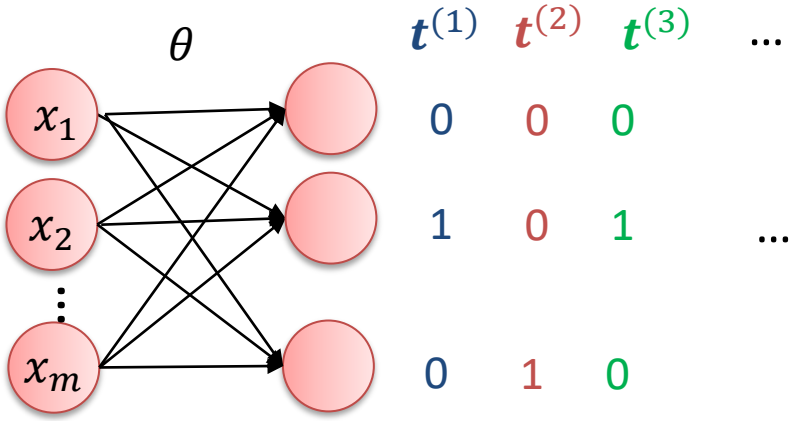
Similarly, $P(\mathbf{t} = (0,1,0)^\top) = 0.5;$
$P(\mathbf{t} = (0,0,1)^\top) = 0.3$

Formulation

- For K-class problems ($K > 2$), we try to learn a Multinoulli distribution $P(\mathrm{t} = k|\boldsymbol{x})$ where $k = 1, \dots, K$
- With one-hot representation $\mathbf{t} = (0, \dots, 1, \dots, 0)^\top$, then

$$P(\mathbf{t}|\boldsymbol{x}) = \Pi_{k=1}^{K} P(\mathrm{t}_k = 1|\boldsymbol{x})^{t_k}$$

A random variable — A value



- Let $P(\mathrm{t}_k = 1|\boldsymbol{x})$ take the following form

$$P(\mathrm{t}_k = 1|\boldsymbol{x}) = \frac{\exp(\boldsymbol{\theta}^{(k)\top}\boldsymbol{x})}{\sum_{j=1}^{K}\exp(\boldsymbol{\theta}^{(j)\top}\boldsymbol{x})} \triangleq h_k(\boldsymbol{x})$$

Clearly, $h_k(\boldsymbol{x}) \in (0,1)$ and $\sum_{k=1}^{K} h_k(\boldsymbol{x}) = 1$

Softmax function

$$h_k(\boldsymbol{x}) = P(\mathrm{t}_k = 1|\boldsymbol{x}) = \frac{\exp(\boldsymbol{\theta}^{(k)\top}\boldsymbol{x})}{\sum_{j=1}^{K}\exp(\boldsymbol{\theta}^{(j)\top}\boldsymbol{x})}$$

- The following function is called *softmax* function

$$\psi(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} = \frac{\exp(z_i)}{\exp(z_i) + \sum_{j\neq i}\exp(z_j)} \in (0,1)$$

- If $z_i > z_j$ for all $j \neq i$
  - Then $\psi(z_i) > \psi(z_j)$ for all $j \neq i$ but it is smaller than 1
- If $z_i \gg z_j$ for all $j \neq i$,
  - then $\psi(z_i) \to 1$ and $\psi(z_j) \to 0$ for $j \neq i$.

Maximum conditional likelihood

- Since

$$P(\mathbf{t}|\boldsymbol{p}) = \Pi_{k=1}^{K} P(\mathrm{t}_k = 1)^{t_k}$$

- Given a dataset $\{(\boldsymbol{x}^{(1)}, \boldsymbol{t}^{(1)}), \ldots, (\boldsymbol{x}^{(N)}, \boldsymbol{t}^{(N)})\}$. The conditional likelihood function (independence assumption):

$$P(\boldsymbol{t}^{(1)}, \ldots, \boldsymbol{t}^{(N)} | \boldsymbol{X}) = \Pi_{n=1}^{N} \Pi_{k=1}^{K} P(\mathrm{t}_k^{(n)} = 1 | \boldsymbol{x}^{(n)})^{t_k^{(n)}}$$

- Maximizing this likelihood function is equivalent to minimizing

Take log and negative, then minimize

$$E(\boldsymbol{\theta}) = -\frac{1}{N} \ln P(\boldsymbol{t}^{(1)}, \ldots, \boldsymbol{t}^{(N)})$$

$$= -\frac{1}{N} \sum_{n=1}^{N} \sum_{k=1}^{K} t_k^{(n)} \ln \frac{\exp(\boldsymbol{\theta}^{(k)\top} \boldsymbol{x}^{(n)})}{\sum_{j=1}^{K} \exp(\boldsymbol{\theta}^{(j)\top} \boldsymbol{x}^{(n)})} \leftarrow h_k^{(n)}$$

Cross-entropy function

$$= -\frac{1}{N} \sum_{n=1}^{N} \sum_{k=1}^{K} t_k^{(n)} \ln h_k^{(n)} = -\frac{1}{N} \sum_{n=1}^{N} \ln P(\mathrm{t}_k^{(n)} = 1 | \boldsymbol{x}^{(n)})$$

Cross-entropy (CE) function

$$E(\boldsymbol{\theta}) = -\frac{1}{N}\sum_{n=1}^{N}\sum_{k=1}^{K} t_k^{(n)} \ln \frac{\exp(\boldsymbol{\theta}^{(k)\top}\boldsymbol{x}^{(n)})}{\sum_{j=1}^{K}\exp(\boldsymbol{\theta}^{(j)\top}\boldsymbol{x}^{(n)})}$$

$$= -\frac{1}{N}\sum_{n=1}^{N} \ln P(\mathrm{t}_k^{(n)} = 1|\boldsymbol{x}^{(n)})$$

where $\boldsymbol{\theta} = \begin{bmatrix} | & | & | & | \\ \boldsymbol{\theta}^{(1)} & \boldsymbol{\theta}^{(2)} & \dots & \boldsymbol{\theta}^{(K)} \\ | & | & | & | \end{bmatrix}^{\top}$.

Why is it called “cross-entropy” function?

Watch the supplementary video uploaded to WebLearning

单选题 1分

设置

What’ s the minimum of the CE function

$$E(\boldsymbol{\theta}) = -\frac{1}{N}\ln P(\boldsymbol{t}^{(1)},\ldots,\boldsymbol{t}^{(N)})$$

$$= -\frac{1}{N}\sum_{n=1}^{N}\sum_{k=1}^{K} t_k^{(n)} \ln \frac{\exp(\boldsymbol{\theta}^{(k)\top}\boldsymbol{x}^{(n)})}{\sum_{j=1}^{K}\exp(\boldsymbol{\theta}^{(j)\top}\boldsymbol{x}^{(n)})}$$

A 0

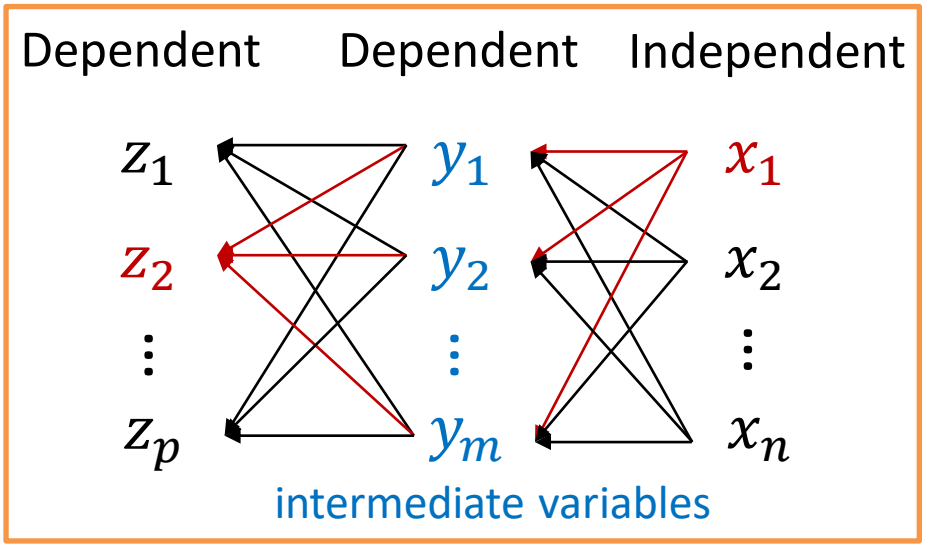
B 1

C $-\infty$

提交

Recap: Derivative of two-step composition

- Independent variables $x_1, x_2, \dots, x_n$
- Each y_i is a function of $x_1, x_2, \dots, x_n$
- Each z_i is a function of $y_1, y_2, \dots, y_m$



What's partial derivative of z_i w.r.t. x_j?

$$\frac{\partial z_i}{\partial x_j} = \sum_{k=1}^{m} \frac{\partial z_i}{\partial y_k} \frac{\partial y_k}{\partial x_j}$$

Sum over the intermediate variables

for any $i \in \{1,2, \dots, p\}$ and $j \in \{1,2, \dots, n\}$

Calculate the gradient

$$E(\boldsymbol{\theta}) = \frac{1}{N}\sum_{n=1}^{N} E^{(n)}(\boldsymbol{\theta}), \quad E^{(n)}(\boldsymbol{\theta}) = -\sum_{i=1}^{K} t_i^{(n)} \ln h_i^{(n)}$$

where $h_i^{(n)} = P(\mathrm{t}_i^{(n)} = 1 | \boldsymbol{x}^{(n)}) = \dfrac{\exp(u_i^{(n)})}{\sum_{j=1}^{K} \exp(u_j^{(n)})}, \quad u_k^{(n)} = \boldsymbol{\theta}^{(k)\top} \boldsymbol{x}^{(n)}$

$$\frac{\partial E^{(n)}}{\partial \boldsymbol{\theta}^{(k)}} = \frac{\partial E^{(n)}}{\partial u_k^{(n)}} \frac{\partial u_k^{(n)}}{\partial \boldsymbol{\theta}^{(k)}} = \sum_{i=1}^{K} \frac{\partial E^{(n)}}{\partial h_i^{(n)}} \frac{\partial h_i^{(n)}}{\partial u_k^{(n)}} \frac{\partial u_k^{(n)}}{\partial \boldsymbol{\theta}^{(k)}}$$

Local sensitivity or local gradient

$$\frac{\partial E^{(n)}}{\partial h_i^{(n)}} = -t_i^{(n)} \frac{1}{h_i^{(n)}}$$

?

$$\frac{\partial u_k^{(n)}}{\partial \boldsymbol{\theta}^{(k)}} = \boldsymbol{x}^{(n)}$$

Calculate the gradient

$$E(\boldsymbol{\theta}) = \frac{1}{N}\sum_{n=1}^{N} E^{(n)}(\boldsymbol{\theta}), \quad E^{(n)}(\boldsymbol{\theta}) = -\sum_{i=1}^{K} t_i^{(n)} \ln h_i^{(n)}$$

where $h_i^{(n)} = P(\mathrm{t}_i^{(n)} = 1 | \boldsymbol{x}^{(n)}) = \dfrac{\exp(u_i^{(n)})}{\sum_{j=1}^{K} \exp(u_j^{(n)})}, \quad u_k^{(n)} = \boldsymbol{\theta}^{(k)\top} \boldsymbol{x}^{(n)}$

If $k \neq i$, u_k appears only in the denominator

$$\frac{\partial h_i^{(n)}}{\partial u_k^{(n)}} =$$

If $k = i$, u_k appears in both numerator and denominator

$$\frac{\partial h_i^{(n)}}{\partial u_k^{(n)}} =$$

Calculate the gradient

$$E(\boldsymbol{\theta}) = \frac{1}{N}\sum_{n=1}^{N} E^{(n)}(\boldsymbol{\theta}), \quad E^{(n)}(\boldsymbol{\theta}) = -\sum_{i=1}^{K} t_i^{(n)} \ln h_i^{(n)}$$

where $h_i^{(n)} = P(\mathrm{t}_i^{(n)} = 1|\boldsymbol{x}^{(n)}) = \dfrac{\exp(u_i^{(n)})}{\sum_{j=1}^{K}\exp(u_j^{(n)})}, \quad u_k^{(n)} = \boldsymbol{\theta}^{(k)\top}\boldsymbol{x}^{(n)}$

If $k \neq i$, u_k appears only in the denominator

$$\frac{\partial h_i^{(n)}}{\partial u_k^{(n)}} = -\frac{\exp(u_i^{(n)})\exp(u_k^{(n)})}{\left(\sum_j \exp(u_j^{(n)})\right)^2} = -h_k^{(n)} h_i^{(n)}$$

If $k = i$, u_k appears in both numerator and denominator

$$\frac{\partial h_i^{(n)}}{\partial u_k^{(n)}} = \frac{\exp(u_k^{(n)})}{\sum_j \exp(u_j^{(n)})} - \frac{\left(\exp(u_k^{(n)})\right)^2}{\left(\sum_j \exp(u_j^{(n)})\right)^2} = h_k^{(n)}(1 - h_k^{(n)})$$

Therefore $\dfrac{\partial h_i^{(n)}}{\partial u_k^{(n)}} = h_i^{(n)}(\Delta_{i,k} - h_k^{(n)})$ where $\Delta_{i,k} = \begin{cases} 1, & \text{if } i = k \\ 0, & \text{else.} \end{cases}$

Calculate the gradient

$$E(\boldsymbol{\theta}) = \frac{1}{N}\sum_{n=1}^{N} E^{(n)}(\boldsymbol{\theta}), \quad E^{(n)}(\boldsymbol{\theta}) = -\sum_{i=1}^{K} t_i^{(n)} \ln h_i^{(n)}$$

where $h_i^{(n)} = P(\mathrm{t}_i^{(n)} = 1 | \boldsymbol{x}^{(n)}) = \dfrac{\exp(u_i^{(n)})}{\sum_{j=1}^{K} \exp(u_j^{(n)})}, \quad u_k^{(n)} = \boldsymbol{\theta}^{(k)\top} \boldsymbol{x}^{(n)}$

$$\frac{\partial E^{(n)}}{\partial \boldsymbol{\theta}^{(k)}} = \sum_{i=1}^{K} \frac{\partial E^{(n)}}{\partial h_i^{(n)}} \frac{\partial h_i^{(n)}}{\partial u_k^{(n)}} \frac{\partial u_k^{(n)}}{\partial \boldsymbol{\theta}^{(k)}}$$

$$= \sum_{i=1}^{K} \left(-t_i^{(n)} \frac{1}{h_i^{(n)}} \right) \left(h_i^{(n)} (\Delta_{i,k} - h_k^{(n)}) \right) \left(\boldsymbol{x}^{(n)} \right)$$

$$= -\left(\sum_{i=1}^{K} t_i^{(n)} \Delta_{i,k} - \sum_{i=1}^{K} t_i^{(n)} h_k^{(n)} \right) \boldsymbol{x}^{(n)}$$

$$= -\left(t_k^{(n)} - h_k^{(n)} \right) \boldsymbol{x}^{(n)}$$

$\sum_{i=1}^{K} t_i^{(n)} = 1$

Calculate the gradient

$$E(\boldsymbol{\theta}) = \frac{1}{N}\sum_{n=1}^{N} E^{(n)}(\boldsymbol{\theta}), \quad E^{(n)}(\boldsymbol{\theta}) = -\sum_{i=1}^{K} t_i^{(n)} \ln h_i^{(n)}$$

where $h_i^{(n)} = P(\mathrm{t}_i^{(n)} = 1|\boldsymbol{x}^{(n)}) = \dfrac{\exp(u_i^{(n)})}{\sum_{j=1}^{K}\exp(u_j^{(n)})}, \quad u_k^{(n)} = \boldsymbol{\theta}^{(k)\top}\boldsymbol{x}^{(n)}$

$$\frac{\partial E^{(n)}}{\partial \boldsymbol{\theta}^{(k)}} = \delta_k^{(n)}\boldsymbol{x}^{(n)}, \text{ where } \delta_k^{(n)} \triangleq \frac{\partial E^{(n)}}{\partial u_k^{(n)}} = -\left(t_k^{(n)} - h_k^{(n)}\right)$$

is the local gradient or local sensitivity.

Average over samples

$$\frac{\partial E}{\partial \boldsymbol{\theta}^{(k)}} = \frac{1}{N}\sum_{n=1}^{N}\frac{\partial E^{(n)}}{\partial \boldsymbol{\theta}^{(k)}} = -\frac{1}{N}\sum_{n=1}^{N}\left(t_k^{(n)} - h_k^{(n)})\right)\boldsymbol{x}^{(n)}$$

$$= -\frac{1}{N}\sum_{n=1}^{N}\left(t_k^{(n)} - P(t_k^{(n)} = 1|\boldsymbol{x}^{(n)})\right)\boldsymbol{x}^{(n)}$$

Vector-matrix form

- Note the definitions

$$\boldsymbol{\theta} = \begin{pmatrix} \theta_{11} & \cdots & \theta_{1m} \\ \vdots & \vdots & \vdots \\ \theta_{K1} & \cdots & \theta_{Km} \end{pmatrix} \quad \frac{\partial E}{\partial \boldsymbol{\theta}} = \begin{pmatrix} \partial E/\partial \theta_{11} & \cdots & \partial E/\partial \theta_{1m} \\ \vdots & \vdots & \vdots \\ \partial E/\partial \theta_{K1} & \cdots & \partial E/\partial \theta_{Km} \end{pmatrix}$$

m: the number of inputs; K: the number of outputs

- Output: $\boldsymbol{f}(\boldsymbol{x})$ is the softmax function where $\boldsymbol{f}, \boldsymbol{b} \in R^K$, $\boldsymbol{x} \in R^m$
- The gradient of the cross-entropy error function

$$\nabla_{\boldsymbol{\theta}} E = \begin{pmatrix} (\partial E/\partial \boldsymbol{\theta}^{(1)})^\top \\ \vdots \\ (\partial E/\partial \boldsymbol{\theta}^{(K)})^\top \end{pmatrix} = \frac{1}{N} \sum_{n=1}^{N} \left(\boldsymbol{f}\left(\boldsymbol{x}^{(n)}\right) - \boldsymbol{t}^{(n)} \right) \left(\boldsymbol{x}^{(n)}\right)^\top \in R^{K \times m}$$

Training and testing

- Calculate the gradient of the cross-entropy error function

$$\nabla_{\boldsymbol{\theta}} E = \frac{1}{N} \sum_{n=1}^{N} \left(\boldsymbol{f}\left(\boldsymbol{x}^{(n)}\right) - \boldsymbol{t}^{(n)} \right) \left(\boldsymbol{x}^{(n)}\right)^{\mathsf{T}}$$

- As before, some regularization term can be incorporated into the cost function

$$J(\boldsymbol{\theta}) = E(\boldsymbol{\theta}) + \lambda \left|\left|\boldsymbol{\theta}\right|\right|^2 / 2$$

- Training: minimize the cost function with gradient $\nabla J(\boldsymbol{\theta})$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla J(\boldsymbol{\theta})$$

  where α is the learning rate

- Testing: find the maximum $P(\mathrm{t}_k = 1 | \boldsymbol{x})$ among k for a new input $\boldsymbol{x}$

Recall: Stochastic gradient decent (SGD)



- Minimizing the cost function over the entire training set is computationally expensive
- Decompose the training set into minibatches and optimize the cost function $L(\boldsymbol{X}^{(i)}, \boldsymbol{y}^{(i)}; \boldsymbol{\theta})$ defined over individual minibatches $(\boldsymbol{X}^{(i)}, \boldsymbol{y}^{(i)})$

  - $J(\boldsymbol{\theta}) = \sum_{i=1}^{N'} L(\boldsymbol{X}^{(i)}, \boldsymbol{y}^{(i)}; \boldsymbol{\theta})$
  - The batchsize ranges from 1 to a few hundreds

- At every iteration, update θ as follows

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \boldsymbol{g}'$$
$$\boldsymbol{g}' = \nabla_{\boldsymbol{\theta}} L(\boldsymbol{X}^{(i)}, \boldsymbol{y}^{(i)}; \boldsymbol{\theta})$$

Introducing bias

- So far we have assumed

$$h_k(\boldsymbol{x}) = P(\mathrm{t}_k = 1|\boldsymbol{x}) = \frac{\exp(u_k^{(n)})}{\sum_{j=1}^{K}\exp(u_j^{(n)})} \qquad u_k^{(n)} = \boldsymbol{\theta}^{(k)\top}\boldsymbol{x}^{(n)}$$

- Sometimes a bias is introduced into $u_k^{(n)}$ and the parameters become $\{\boldsymbol{W}, \boldsymbol{b}\}$

$$u_k^{(n)} = \boldsymbol{w}^{(k)\top}\boldsymbol{x}^{(n)} + b^{(k)}$$

- It's easy to show that

$$\frac{\partial E}{\partial \boldsymbol{w}^{(k)}} = -\frac{1}{N}\sum_{n=1}^{N}\left(t_k^{(n)} - h_k(\boldsymbol{x}^{(n)})\right)\boldsymbol{x}^{(n)}$$

$$\frac{\partial E}{\partial b^{(k)}} = -\frac{1}{N}\sum_{n=1}^{N}\left(t_k^{(n)} - h_k(\boldsymbol{x}^{(n)})\right)$$

- Regularization is often applied on $\boldsymbol{W}$ only Why?

$$J(\boldsymbol{W}, \boldsymbol{b}) = E(\boldsymbol{W}, \boldsymbol{b}) + \lambda\left|\left|\boldsymbol{W}\right|\right|^2/2$$

Softmax is over-parameterized

- The hypothesis

$$h_k(\boldsymbol{x}) = P(\mathrm{t}_k = 1|\boldsymbol{x}) = \frac{\exp(\boldsymbol{\theta}^{(k)\top}\boldsymbol{x})}{\sum_{j=1}^{K}\exp(\boldsymbol{\theta}^{(j)\top}\boldsymbol{x})} = \frac{\exp((\boldsymbol{\theta}^{(k)}-\boldsymbol{\phi})^{\top}\boldsymbol{x})}{\sum_{j=1}^{K}\exp((\boldsymbol{\theta}^{(j)}-\boldsymbol{\phi})^{\top}\boldsymbol{x})}$$

Then the new parameters $\widehat{\boldsymbol{\theta}}^{(k)} \equiv \boldsymbol{\theta}^{(k)} - \boldsymbol{\phi}$ will result in the same prediction

- Minimizing the cross-entropy function has infinite number of solutions since

$$E(\boldsymbol{\theta}) = -\sum_{n=1}^{N}\sum_{k=1}^{K} t_k^{(n)} \ln \frac{\exp(\boldsymbol{\theta}^{(k)\top}\boldsymbol{x}^{(n)})}{\sum_{j=1}^{K}\exp(\boldsymbol{\theta}^{(j)\top}\boldsymbol{x}^{(n)})} = E(\boldsymbol{\theta} - \boldsymbol{\Phi})$$

where $\boldsymbol{\Phi} = (\boldsymbol{\phi}, ..., \boldsymbol{\phi})$

Relationship between logistic regression and softmax regression

Logistic regression:

$$P(\mathrm{t}=1|\boldsymbol{x}) = \frac{1}{1+\exp(-\theta^\top \boldsymbol{x})}$$

$$P(\mathrm{t}=0|\boldsymbol{x}) = 1 - P(t=1|\boldsymbol{x})$$

Softmax regression:

$$P(\mathrm{t}_k=1|\boldsymbol{x}) = \frac{\exp(\boldsymbol{\theta}^{(k)\top}\boldsymbol{x})}{\sum_{j=1}^{K}\exp(\boldsymbol{\theta}^{(j)\top}\boldsymbol{x})}$$

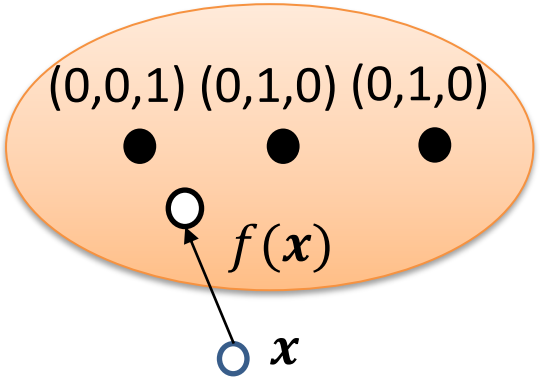
Is there any relationship between them?



Watch the supplementary video uploaded to WebLearning

Summary of part 2

Function mapping



Linear regression → nonlinear regression

$\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{W}^{\top}\boldsymbol{x} + \boldsymbol{b}$

$\boldsymbol{f}(x) = \boldsymbol{g}(\boldsymbol{W}^{\top}\boldsymbol{x} + \boldsymbol{b})$

MLE

1. normal distribution:
   - $\boldsymbol{g}(\cdot)$ is any nonlinear function
   - Loss function: MSE
2. multinoulli distribution:
   - $\boldsymbol{g}(\cdot)$ is softmax function
   - Loss function: Cross-entropy

Softmax regression!

单选题 1分

设置

For a 3-class classification problem, if we want to use the cross-entropy function as the loss function, which of the following shall we choose as the classifier?

A Logistic sigmoid function

B Softmax function

C Gaussian function

D Any nonlinear function

提交

单选题 1分

设置

If we use a nonlinear function $\boldsymbol{g}(\boldsymbol{z}) = \max(0, \boldsymbol{z})$ as the classifier, can we use the cross-entropy function as the loss function?

A Yes

B No

提交

Outline

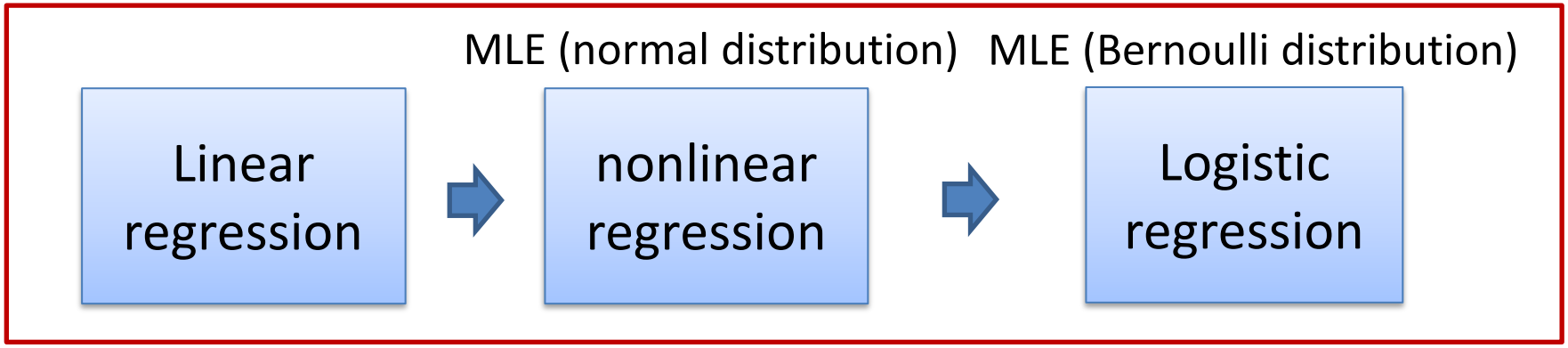
I. Logistic regression

II. Softmax regression

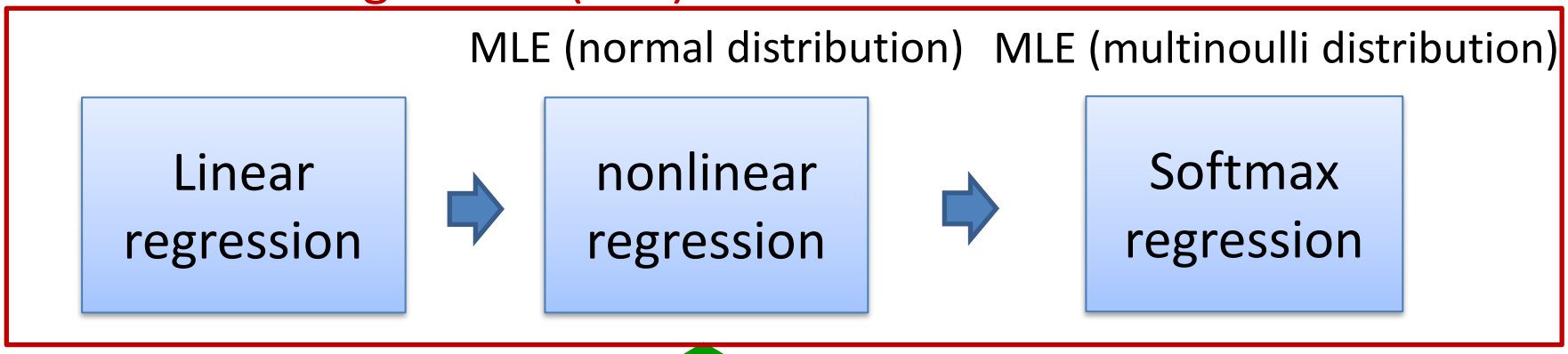
III. Summary

Summary of this lecture

I. Logistic regression (K=2)



II. Softmax regression (K>2)



If the task is regression,
we stop here

Summary of this lecture

- Nonlinear regression (linear regression as a special case)
  - Output: $\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{h}(\boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{x})$, where $\boldsymbol{h}$ could be any act function
  - MSE: $E = \frac{1}{N}\sum_{n=1}^{N} E^{(n)}$, $E^{(n)} = \frac{1}{2}\left\|\boldsymbol{h}\left(\boldsymbol{x}^{(n)}\right) - \boldsymbol{t}^{(n)}\right\|_2^2$
  - Gradient: $\nabla_{\boldsymbol{\theta}} E = \frac{1}{N}\sum_{n=1}^{N}\left(\boldsymbol{f}\left(\boldsymbol{x}^{(n)}\right) - \boldsymbol{t}^{(n)}\right) \odot \boldsymbol{f}'\left(\boldsymbol{x}^{(n)}\right)\left(\boldsymbol{x}^{(n)}\right)^{\mathsf{T}}$
- Softmax regression (logistic regression as a special case)
  - Output: $\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{h}(\boldsymbol{\theta}\boldsymbol{x})$, where $\boldsymbol{h}$ is the softmax function
  - Cross-entropy error: $E = \frac{1}{N}\sum_{n=1}^{N} E^{(n)}$, $E^{(n)} = -\left(\boldsymbol{t}^{(n)}\right)^{\mathsf{T}} \ln \boldsymbol{h}^{(n)}$
  - Gradient: $\nabla_{\boldsymbol{\theta}} E = \frac{1}{N}\sum_{n=1}^{N}\left(\boldsymbol{f}\left(\boldsymbol{x}^{(n)}\right) - \boldsymbol{t}^{(n)}\right)\left(\boldsymbol{x}^{(n)}\right)^{\mathsf{T}}$

Video material

- Watch the supplementary video uploaded to WebLearning
  - Relationship between softmax regression and logistic regression
  - Cross-entropy in general

Homework 1

- Softmax for MNIST classification
  - Deadline: Oct 13 (Monday noon) 12:00